

AFRILANG Stdlib

std/c/gamealphabet

Bibliothèque AFRILANG `std/c/gamealphabet` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : alphabet,

```
`import "std/c/gamealphabet" · `use gamealphabet`
```

- `alphabet(board list number, depth number, alpha number, beta number, maximizing bool) ? number`
- `bestMove(board list number, depth number) ? number`
- `pruneCount(board list number, depth number) ? number`
- `evaluate(board list number) ? number`
- `search(board list number, depth number) ? number`
- `isWinning(board list number) ? bool`
- `windowSearch(board list number, depth number, window number) ? number`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/gamealphabeta` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : alphabeta,

```
`import "std/c/gamealphabeta" . `use gamealphabeta`  
- `alphabeta(board list number, depth number, alpha number, beta number, maximizing bool) ?  
number`  
- `bestMove(board list number, depth number) ? number`  
- `pruneCount(board list number, depth number) ? number`  
- `evaluate(board list number) ? number`  
- `search(board list number, depth number) ? number`  
- `isWinning(board list number) ? bool`  
- `windowSearch(board list number, depth number, window number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/gamealphabeta"  
use gamealphabeta
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? alphabeta(board list number, depth number, alpha number, beta number, maximizing bool) ? number  
? bestMove(board list number, depth number) ? number  
? pruneCount(board list number, depth number) ? number  
? evaluate(board list number) ? number  
? search(board list number, depth number) ? number  
? isWinning(board list number) ? bool  
? windowSearch(board list number, depth number, window number) ? number
```

4. Exemples d'utilisation

```
import "std/c/gamealphabeta"  
use gamealphabeta  
  
create result = alphabeta(/* args */)   
say result  
  
create result = bestMove(/* args */)   
say result  
  
create result = pruneCount(/* args */)   
say result  
  
create result = evaluate(/* args */)   
say result  
  
create result = search(/* args */)   
say result  
  
create result = isWinning(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/gamealphabeta"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module gamealphabeta
function alphabeta(board list number, depth number, alpha number, beta number, maximizing bool)
returns number
end
function bestMove(board list number, depth number) returns number
end
function pruneCount(board list number, depth number) returns number
end
function evaluate(board list number) returns number
end
function search(board list number, depth number) returns number
end
function isWinning(board list number) returns bool
end
function windowSearch(board list number, depth number, window number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/gamealphabeta` (tier complex, category complex). Exports 7 function(s): alphabeta, bestMove, pru

```
`import "std/c/gamealphabeta" . `use gamealphabeta`  
- `alphabeta(board list number, depth number, alpha number, beta number, maximizing bool) ?  
number`  
- `bestMove(board list number, depth number) ? number`  
- `pruneCount(board list number, depth number) ? number`  
- `evaluate(board list number) ? number`  
- `search(board list number, depth number) ? number`  
- `isWinning(board list number) ? bool`  
- `windowSearch(board list number, depth number, window number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/gamealphabeta"  
use gamealphabeta
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? alphabeta(board list number, depth number, alpha number, beta number, maximizing bool) ? number  
? bestMove(board list number, depth number) ? number  
? pruneCount(board list number, depth number) ? number  
? evaluate(board list number) ? number  
? search(board list number, depth number) ? number  
? isWinning(board list number) ? bool  
? windowSearch(board list number, depth number, window number) ? number
```

4. Usage examples

```
import "std/c/gamealphabeta"  
use gamealphabeta  
  
create result = alphabeta(/* args */)   
say result  
  
create result = bestMove(/* args */)   
say result  
  
create result = pruneCount(/* args */)   
say result  
  
create result = evaluate(/* args */)   
say result  
  
create result = search(/* args */)   
say result  
  
create result = isWinning(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/gamealphabeta"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module gamealphabeta
function alphabeta(board list number, depth number, alpha number, beta number, maximizing bool)
returns number
end
function bestMove(board list number, depth number) returns number
end
function pruneCount(board list number, depth number) returns number
end
function evaluate(board list number) returns number
end
function search(board list number, depth number) returns number
end
function isWinning(board list number) returns bool
end
function windowSearch(board list number, depth number, window number) returns number
end
end
```